



DISEÑO DIGITAL 2

MEMORIA DEL DISEÑO:

MEDT: Medidor de Temperatura

Autor: María Sancho
Dayana Román
David Heredero
Pablo Bosom
Jesus Gauche

Curso 2024-2025.

Control de versiones

Versión	Fecha	Cambios realizados
0.0	20/04/2025	Puesta en común de la información del proyecto y lluvia de ideas sobre sus niveles de abstracción
1.0	22/04/2025	Creacion inicial del diseño, implementando la interfaz con el generador de las señales SC y CS
1.1	25/04/25	Creación del registro para la implementación de los datos del sensor y su control
1.2	27/04/25	Simulación del primer hito y depurado de errores
2.0	02/05/2025	Desarrollo de los circuitos para calcular el valor de la temperatura en Kelvin y Fahrenheit. Utilizando una logica que permita cambiar entre ellos utilizando el pulsador KEY1 En nuestro caso llamado K1
2.1	06/05/2025	Depuración del sistema de conversión y pruebas en simulación
2.2	08/05/2025	Anexión de gestión de pulsaciones y comprobación
3.0	10/05/2025	Gestión del pulsador K0 y prototipo de funcionamiento de displays
3.1	15/05/2025	Corrección y completado del sistema de displays. Prueba por test bench
3.2	16/05/2025	Integración del proyecto en Quartus.
3.3	21/05/2025	Depuración del circuito en la tarjeta DECA. Cambios en control_spi por fallos y pruebas finales. Diseño concluido

Tabla de contenido

1	Especificación del diseño.	4
1.1	Introducción.....	4
1.2	Interfaces	4
1.2.1	Interfaz con el sensor de temperatura	4
1.2.2	Interfaz con los pulsadores	4
1.2.3	Interfaz con la barra de displays de 7 segmentos	5
1.3	Especificaciones funcionales	5
1.4	Especificaciones no funcionales.....	6
2	Diseño jerárquico.....	8
2.1	Bloque SPI.....	8
2.2	Bloque CONVERTOR TEMPERATURA.....	10
2.3	Bloque GESTOR PULSADORES	11
2.4	Bloque DISPLAYS	¡Error! Marcador no definido.
3	Diseño detallado	12
3.1	Estructura del proyecto	12
3.2	Jerarquía del diseño y diseño detallado	13
4	Pruebas de verificación funcional de MEDTH.....	15
4.1	Test del SPI.....	15
4.2	Test del CONVERTOR TEMPERATURA.....	15
4.3	Test del PULSADOR	15
4.4	Test del diseño completo.	¡Error! Marcador no definido.
5	Diseño físico	16
5.1	Asignación de pines.....	16
5.2	Restricciones de la síntesis	17
5.3	Recursos utilizados	17
5.4	Frecuencia máxima de reloj.....	18
6	Prototipado	18
7	Bibliografía.....	19
8	APÉNDICE 1. Interfaz FPGA-sensor. Compatibilidad lógica y MR	¡Error! Marcador no definido.
9	APÉNDICE 2. Secuencia de operaciones en la interfaz I2C .	¡Error! Marcador no definido.

1 Especificación del diseño.

1.1 *Introducción*

El proyecto MEDT (Medidor de Temperatura) tiene como objetivo desarrollar un sistema digital que permita medir la temperatura utilizando el sensor de temperatura LM71. Este sistema permitirá a los usuarios seleccionar entre tres unidades de temperatura: grados centígrados (°C), Fahrenheit (°F) y Kelvin (K), y visualizar la medida a través de una serie de displays BCD (visualización en decimal), con 1°C de precisión. Además, el sistema será capaz de actualizar la medición de la temperatura en intervalos configurables entre 2, 4, 6 y 8 segundos. El sistema MEDT emplea dos pulsadores disponibles en la tarjeta DECA para permitir la interacción del usuario. El pulsador izquierdo (KEY1) se utiliza para cambiar entre las unidades de temperatura, mientras que el pulsador derecho (KEY0) controla el periodo de actualización de la medición.

1.2 *Interfaces*

El sistema se interconecta al sensor de temperatura hexadecimal y a una barra de displays de 7 segmentos, además de los propios pulsadores de la tarjeta; a través de sendas interfaces que se describen a continuación.

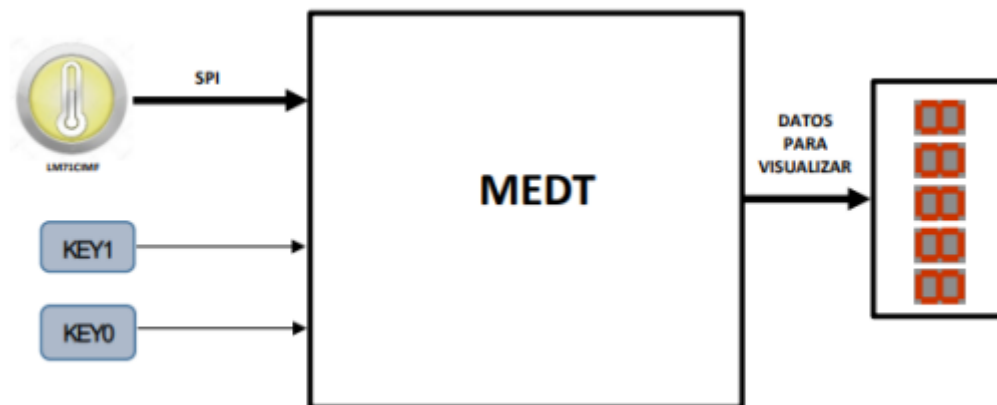


Fig. 1. interconexion del MEDT al sensor de temp con dos pulsadores y barra de 7 seg

1.1.1 1 Interfaz con el sensor de temperatura

El sistema se comunica con el sensor de temperatura por medio de una interfaz SPI

Señal	Dirección	Descripción
CS	Salida	Señal del chip-select de la interfaz SPI
SC	Salida	Señal de reloj de la interfaz SPI
SIO	Entrada	Señal de datos de la interfaz SPI

1.1.2 Interfaz con pulsadores

El sistema se controla con dos pulsadores pull-up.

Señal	Dirección	Descripción
K0	entrada	Pulsador
K1	entrada	Pulsador

1.1.3 Interfaz con la barra de displays de 7 segmentos

El sistema realiza la presentación de los datos utilizando una barra de displays de 7 segmentos del tipo cátodo común. La interfaz es la siguiente :

Señal	Dirección	Descripción
disp[7..0]	salida	disp [0] : segmento g disp [1] : segmento f disp [2] : segmento e disp [3] : segmento d disp [4] : segmento c disp [5] : segmento b disp [6] : segmento a disp [7] : segmento punto
mux_disp[7..0]	salida	mux_disp [0] : cátodo del display 0 (LSD) mux_disp [1] : cátodo del display 1 mux_disp [2] : cátodo del display 2 mux_disp [3] : cátodo del display 3 mux_disp [4] : cátodo del display 4 mux_disp [5] : cátodo del display 5 mux_disp [6] : cátodo del display 6 mux_disp [7] : cátodo del display 7 (MSD)

La interfaz permite iluminar solo un display a la vez. El display se selecciona activando (a nivel bajo) el cátodo correspondiente. El display activo se ilumina de acuerdo al código de 7 segmentos y punto decimal introducido (nivel alto).

1.2 Especificaciones funcionales

Ref	Especificación
ESP00	El sistema realiza medidas de temperatura cada 2, 4, 6 y 8 segundos.
ESP01	El rango de representación de la temperatura estará entre 150 °C y -40 °C, con una resolución de 1 °C.
ESP02	La temperatura podrá representarse en grados centígrados, Fahrenheit o en kelvin.
ESP03	Para el cálculo de los kelvin a partir de los grados centígrados se utilizará la expresión $TKEL = TCENT + 273$
ESP04	Para el cálculo de los grados Fahrenheit a partir de los grados centígrados se utilizará la expresión $TF = 1.8125 \times TCENT + 32$

ESP05	El sistema representará la temperatura en BCD, utilizando 6 displays : En cada uno de ellos se representa lo siguiente : • display 0 (LSD) : letra “C” para centígrados, “F” para Fahrenheit y “°” para kelvin • display 1 : en blanco (no debe lucir ningún segmento) • display 2 : unidades de la temperatura • display 3 : decenas de la temperatura o signo si la temperatura es negativa y mayor que -10 grados • display 4 : centenas de la temperatura o signo si la temperatura es negativa y menor que -9 grados • display 5: en blanco (no debe lucir ningún segmento) • display 6 : indicación del periodo de actualización de temperatura • display 7 : indicación de nueva medida de temperatura “0”
ESP06	Los ceros no significativos de las decenas y centenas no se representarán. El signo menos, en su caso, se ubicará en la posición de las centenas (si su valor en las decenas no es 0) o de las decenas (si el valor de las decenas es 0).
ESP07	Inicialmente la presentación en los displays estará configurada en el modo grados centígrados
ESP08	Será posible modificar el modo de representación de temperatura empleando el pulsador izquierdo (KEY1): cuando se realice una pulsación (activación) se cambiará el modo de representación cíclicamente: de grados centígrados a kelvin, de kelvin a Fahrenheit, de Fahrenheit a centígrados, etcétera.
ESP09	Inicialmente las medidas de temperatura se realizarán con un periodo de 4 segundos
ESP10	Será posible modificar el periodo de medida de temperatura empleando el pulsador derecho (KEY0): cuando se realice una pulsación (activación) se cambiará el periodo cíclicamente: de 2 a 4 segundos, de 4 a 6, de 6 a 8, de 8 a 2, etcétera
ESP11	Cuando se haya tomado una nueva medida de temperatura se indicará mediante un “0” presentada en el display 7 durante un segundo.

1.3 Especificaciones no funcionales

Ref	Especificación
ESP24	El sistema se diseñará utilizando VHDL. Podrán utilizarse como elementos de librería los IPs de Altera que sean necesarios.

ESP25	Se utilizará <i>ModelSim</i> como herramienta de simulación y <i>Quartus Prime</i> como herramienta para la realización del diseño físico,
ESP26	El sistema se prototipará utilizando una tarjeta DECA-MAX10 del fabricante Arrow [2]
ESP27	Para prototipar el sistema se conectará a la tarjeta DECA-MAX10 una tarjeta de expansión [3] con los 8 displays y un conector para el teclado hexadecimal.
ESP28	Se utilizará como fuente de reloj uno de los osciladores de 50MHz que posee la tarjeta DECA-MAX10.

2 Diseño jerárquico

El diagrama de la Fig. 3 representa el primer nivel de la jerarquía del diseño¹:

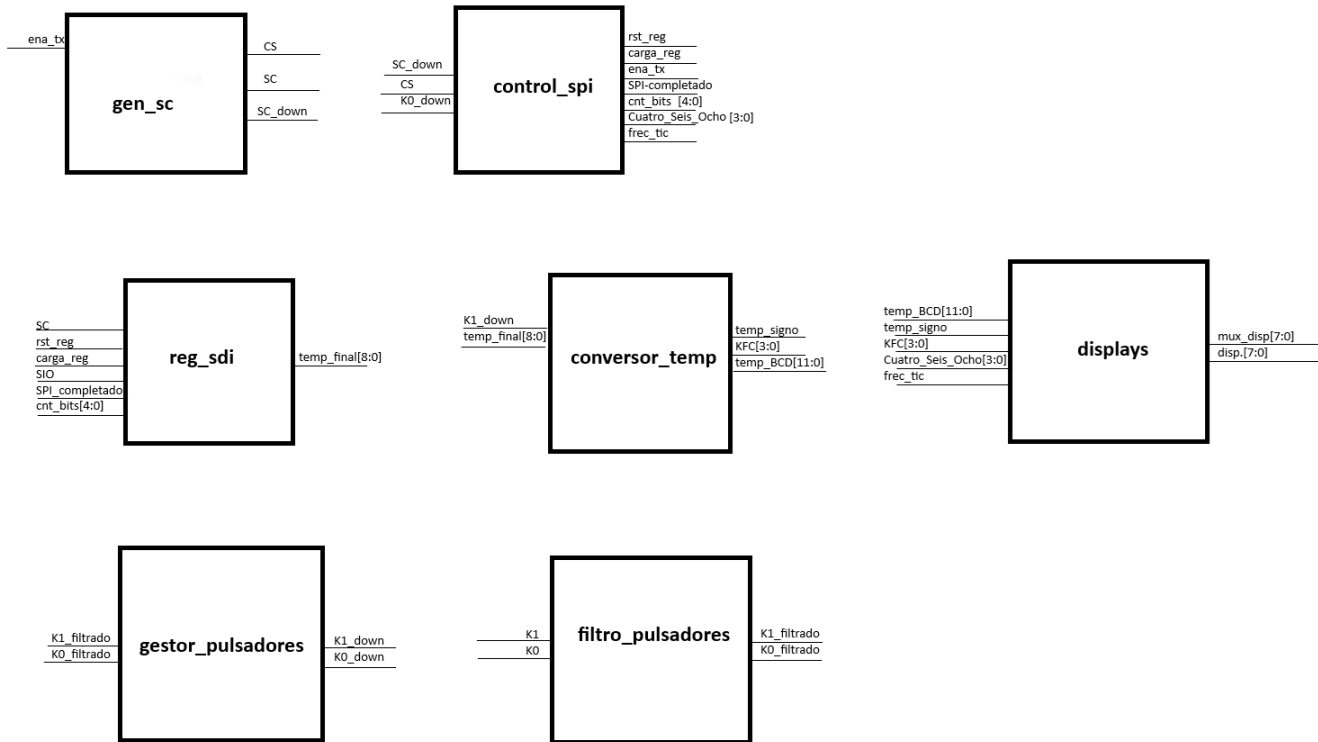


Fig. 3. Diagrama de bloques de la jerarquía de MEDT.

En los siguientes subapartados se describe la interfaz y la función de cada uno de estos bloques.

2.1 Bloque SPI

Este bloque se subdivide en 3 :

- Generador de señal SC

Señal	Dirección	Descripción
SC	entrada	Señal del reloj de la interfaz spi
ena_tx	entrada	Habilitación de transmisión : (p. ej. cada 2 s, 4 s...). Cuando está a nivel alto se habilita la generación de señales
CS	salida	“Chip-select”, se pone en '0' durante la transferencia, en '1' en reposo.
SC_down	salida	Pulso de subida en SC: genera un único pulso ('1') cuando el contador llega a 2 (en bajo)

La señal CS habilita la transferencia de datos, cuando ena_tx está activo; a la par que se generan los pulsos del reloj del sensor

- Control SPI

Señal	Dirección	Descripción
CS	entrada	Señal chip-select (desde gen_sc), usada aquí sólo como indicación de selección.
SC_up	entrada	Pulso de subida de reloj serie (desde gen_sc) que indica flanco de SC alto.
K0_down	entrada	Pulsador para cambiar la frecuencia de medida (de 2 s $\rightarrow$ 4 s $\rightarrow$ 6 s $\rightarrow$ 8 s $\rightarrow$ 2 s).
rst_reg	salida	Reset sincrónico del registro de desplazamiento (reg_spi).
carga_reg	salida	Habilita la carga de cada bit en el registro de desplazamiento en flanco SC_down.
ena_tx	salida	Activa la ventana de transmisión SPI (para gen_sc y reg_spi) durante un periodo de medida.
SPI_completado	salida	Indica fin de recepción de los bits SPI de datos.
cnt_bits	salida	Contador de bits recibidos (0...17) que controla cuándo termina la transferencia SPI.
Cuatro_Seis_Ocho	salida	Indica en qué modo de periodo estamos en Hexadecimal: X'2' (2 s), X'4' (4 s), X'6' (6 s), X'8' (8 s).
frec_tic	salida	Pulso periódico de 2 s, 4 s, 6 s u 8 s . Indica el inicio de una medida de temperatura.

El bloque **control_spi** orquesta la ventana de lectura SPI y la frecuencia de muestreo configurable: genera internamente contadores que producen pulsos de 2 s, 4 s, 6 s y 8 s, seleccionados por una máquina de estados sensible al botón K0_down; cada pulso frec_tic dispara la máquina SPI (INI $\rightarrow$ TX), que alerta ena_tx y resetea el registro, cuenta 17 flancos de SC_up para desplazar bits y al completarse activa SPI_completado, desactiva la transferencia y vuelve a INI, mientras que las salidas rst_reg, carga_reg, ena_tx, cnt_bits, SPI_completado y el vector Cuatro_Seis_Ocho reflejan el progreso y permiten coordinar los bloques vecinos.

- Registro de desplazamiento

Señal	Dirección	Descripción
SC	entrada	Señal del reloj de la interfaz spi
SIO	entrada	Datos serie del sensor (bit a bit).
rst_reg	entrada	Reset sincrónico del registro de desplazamiento.
carga_reg	entrada	Habilitación de captura de bits en el registro (mientras SPI_completado = '0' y cnt_bits > 1).
SPI_completado	entrada	Indicador de fin de transmisión SPI.
cnt_bits	entrada	Contador de bits recibidos (5 bits). Usado para contabilizar los bits que se van recibiendo.
temp_final	salida	Registro de 9 bits (8 datos + signo) que contiene la temperatura final en °C (resolución 1 °C).

Implementa un **registro** de desplazamiento de 16 bits que, tras habilitarse (carga_reg) y mientras no se haya completado la recepción SPI, va introduciendo sucesivamente cada bit de SIO; una vez que la señal SPI_completado se activa, cogemos los 9 bits más altos de ese registro (En complemento a 2, con 9 bits tenemos una **resolución de 1 grado**) se capturan en temp_final, todo ello sincronizado por clk y reiniciable mediante rst_reg o nRst.

2.2 Bloque CONVERSOR TEMPERATURA

Señal	Dirección	Descripción
temp_final	entrada	Temp medida en complemento a 2 con 1 grado de resolución
K1_down	entrada	Pulsador para cambiar las unidades de representación de la temperatura (C -> K -> F)
temp_signo	salida	Bit de signo correspondiente a la temp representada, en función del estado K, F o C en el que se encuentre.
temp_BCD	salida	Temperatura convertida a BCD 3 dígitos (C, D, U) en función del estado K, F o C en el que se encuentre.
KFC	salida	Salida que indica en que unidades estamos

El bloque **conversor_temp** permite rotar entre tres unidades de temperatura (°C, K, °F) al pulsar K1_down, almacenando el estado en estado_KFC y mostrando su código

ASCII en KFC. Según el modo, toma la temperatura con signo de temp_final, la extiende o ajusta sumando la constante (273 para Kelvin, y tras un escalado aproximado 1.8×32 para Fahrenheit), y genera temp_out de 10 bits. Después extrae el bit de signo (temp_signo), calcula el valor absoluto (temp_bin), y convierte ese valor en BCD de tres dígitos con un método basado en restas sucesivas y correcciones (algoritmo Double Dabble simplificado), entregando el resultado en temp_BCD.

2.3 Bloque GESTOR PULSADORES

Al igual que el bloque SPI, este se divide en :

- Filtrado de pulsadores

Señal	Dirección	Descripción
K0	entrada	Pulsador derecho (KEY0) utilizado para cambiar el periodo de actualización de la temp
K1	entrada	Pulsador izquierdo (KEY1) utilizado para cambiar entre la unidad de la temp C K y F
K0_filtrado	salida	Pulsación limpia de K0: se pone a '1' cuando se detecta un flanco estacionario genuino del pulsador.
K1_filtrado	salida	Pulsación limpia de K1: se pone a '1' cuando se detecta un flanco estacionario genuino del pulsador.

El bloque **filtro_pulsadores** : metastabilidad de dos pulsadores usando tres etapas de sincronización y un registro de desplazamiento de 4 muestras invertidas; cuando detecta un cambio estable (mismo nivel durante 4 ciclos de reloj), sitúa en la salida correspondiente (K1_filtrado o K0_filtrado) un pulso alto indicando una pulsación válida.

- Gestor de pulsadores

Señal	Dirección	Descripción
K1_filtrado	entrada	Pulsación limpia de K1.
K0_filtrado	entrada	Pulsación limpia de K0.
K1_down	salida	Un pulso para salida de K1 cuando detecta un flanco de subida
K0_down	salida	Un pulso para salida de K0 cuando detecta un flanco de subida

Detecta un flanco de subida (de 0 a 1, cuando soltamos el pulsador) y lo convierte en una señal de un flanco de reloj a 1.

2.4 Bloque DISPLAYS

Señal	Dirección	Descripción
temp_BCD	entrada	Temperatura en BCD 3 dígitos (U,D,C)
temp_signo	entrada	Bit de signo
KFC	entrada	Indica el modo K, F o C
Cuatro_Seis_Ocho	entrada	Indica en qué modo de periodo estamos en Hexadecimal
frec_tic	entrada	Pulso periódico de 2 s, 4 s, 6 s u 8 s ,indica el inicio de una medida de temperatura.
mux_disp[7:0]	salida	Señal de activación de los displays (cátodos), activa a nivel bajo
disp[7:0]	salida	Código de 7 segs

El módulo del nivel superior del sistema MEDT coordina la operación global del sistema, obteniendo la temperatura, haciendo la conversión a distintas escalas y presentando la información en los displays de 7 seg, también cambia el ritmo de actualización y el modo de visualización,

Al recibir la señal de nRst activa el sistema se resetea. El reloj clk de 50Mhz se utiliza para la generación del tics interno de tic_2_5ms que es utilizado para la multiplexación del display.

La señal temp_BCD contiene la temperatura en formato BCD, representada por tres dígitos unida al bit temp_signo que indica si la temperatura es positiva o negativa.

El vector KFC determina en que unidades se muestra la temperatura, Centígrados(C) Kelvin(K) o Fahrenheit(F). Esta opción la controla el usuario pulsando KEY1 o en el código llamada K1

La señal Cuatro_Seis_Ocho codifica el intervalo de tiempo entre cada nueva medición (2,4,6,8s). El usuario podrá cambiar manualmente con el KEY0 en el código llamado K0.

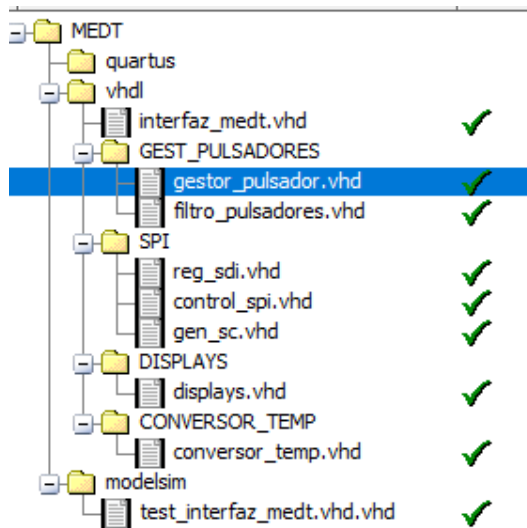
- Multiplexación y visualización:
El sistema genera señales de control mux_disp para activar secuencialmente cada uno de los 8 displays de 7 seg. La información a mostrar en cada display se transmite mediante disp.
Cada vez que se quiere una nueva medida de temperatura se representa un “0” en el display de 7 seg durante 1s como indicador visual de que se está actualizando como indica la especificación.

3 Diseño detallado

3.1 Estructura del proyecto

El proyecto está almacenado en la carpeta MEDT, que a su vez contiene las carpetas hdl, modelsim y quartus. La carpeta hdl contiene los ficheros RTL y estructural del diseño. La carpeta modelsim contiene el proyecto de simulación (MEDT.mpf), y el fichero donde se

define el testbench. Finalmente, *quartus* contiene el proyecto para el diseño físico y los ficheros relacionados con éste. La siguiente figura muestra la jerarquía de carpetas dentro de la carpeta MEDT:



3.2

Fig. 4. Estructura de carpetas del proyecto.

3.3 Jerarquía del diseño y diseño detallado

En la siguiente tabla se muestra la jerarquía del diseño y se realiza una descripción somera de los módulos que la componen. Los detalles del funcionamiento de cada módulo se han documentado en comentarios en la cabecera de los propios módulos

Rojo : descripciones estructurales, Azul : descripciones RTL, Verde : descripciones mixtas (RTL con algún módulo instanciado), Negro : IPs de Altera

Nivel jerárquico					Descripción
1 (top)	2	3	4	5	
Interfaz_medt					Medidor de temperatura
	SPI				Interfaz SPI. Nivel superior de la jerarquía del circuito. Instancia una interfaz de programación y un circuito de comunicaciones compatible SPI.
		gen_sc			Genera la señal de reloj SC de la interfaz SPI
		control_spi			Controlador de la interfaz SPI, encargada de iniciar la transmisión y generar 2,4,6 y 8s de cada medida.
		reg_spi			Lee en serie de los datos que le llegan del sensor de temperatura y los guarda.
	Gestor_Pulsadores				
		gestor_pulsador			Controlador de pulsadores. Codifica cada pulsador filtrado que viene desde filtro_pulsadores y detecta a nivel bajo la pulsación
		filtro_pulsadores			Filtrado de los pulsadores.
	Conversor_Temp				Conversor y controlador de la temperatura
		conversor_temp			Gestión de la conversión de temperatura que le llega desde reg_sdi. Codifica los datos obtenidos en BCD de 3 dígitos incluyendo signo, 3 modos de conversión K (Kelvin), F (Fahrenheit), C° (Celsius).
	Displays				Controlador de displays. Interpreta los comandos del teclado e identifica el modo de visualización. Multiplexor de datos. Visualiza los datos en la barra de displays en función del modo de visualización. Utiliza multiplexión temporal de los datos.

4 Pruebas de verificación funcional de MEDTH

El plan de pruebas de MEDTH consiste en un conjunto de tests dirigidos. Se realizan tests específicos para los bloques funcionales más complejos, tests en los que se combinan varios bloques funcionales cuando resulta necesario y un test final para el diseño completo. Algunas de las pruebas requieren escalado temporal para reducir su duración.

En los siguientes subapartados se describen los tests realizados.

4.1 Test del SPI

Ubicación de los ficheros del test	MEDT/modelsim	
Simulación escalada	Seguimos las medidas a partir de tic medida cada 3000 ciclos	
Ficheros	test_interfaz_spi	Test-bench con las señales del sensor
Descripción del test	Metemos varios ciclos de tic y observamos las señales correspondiente al gen_sc y al reg_sdi ; ayudandonos de algunas señales de control. Debemos guardar el dato de SDAT en el registro durante los flancos de bajada de SC.	

4.2 Test del CONVERTOR DE TEMPERATURA

Ubicación de los ficheros del test	MEDT/modelsim/test_interfaz_medt	
Simulación escalada	tic de 2.5ms	
Ficheros	test_interfaz_spi	Test-bench con las señales del sensor
Descripción del test	Comprobamos de 150° a -40°. Simulamos cambios de botón K1 para modificar las unidades y comparamos la salida en binario de temp_final (del registro) con los valores vistos y calculados manualmente	

4.3 Test de los PULSADORES

Ubicación de los ficheros del test	MEDT/modelsim/test_interfaz_medt	
Simulación escalada	tic de 20ms	
Ficheros	test_interfaz_spi	Test-bench con las señales del sensor
Descripción del test	Metemos varios ciclos de reloj activando y desactivando los pulsadores K0 y K1 y esperamos tres medidas	

4.4 Test de DISPLAYS

Ubicación de los ficheros del test	MEDT/modelsim/test_interfaz_medt	
Simulación escalada	tic de 20ms	

Ficheros	test_interfaz_spi	Test-bench con las señales del sensor
Descripción del test	Con todo el sistema en funcionamiento observamos el valor de la temperatura en BCD y las señales de disp. Y mux_disp. Vemos si los valores coinciden con lo que debe verse en cada display flanco a flanco	

5 Diseño físico

En este apartado se documentan los detalles básicos relacionados con el diseño físico del circuito: la asignación de pines, las restricciones de la síntesis y los informes que proporciona Quartus Prime sobre los recursos de la FPGA utilizados y la frecuencia máxima de reloj obtenida.

5.1 *Asignación de pines*

En la siguiente tabla se detalla la asignación de los pines de la interfaz de MEDTH a los pines de la FPGA especificando para cada caso el tipo de pin, el número de pin de la FPGA que se corresponde con el pin del diseño, el banco al que corresponde, el estándar de entrada/salida que utiliza, el fan-out (para los pines de salida) y el slew-rate.

Pin de la interfaz del diseño	direccion	Pin FPGA	I/O bank	I/O standard	Current strength (mA)	Slew rate	Pull-up interno
clk	Input	M8	2	2.5-V	12	2	No
rst_n	Input	J22	6	1.5-V Schmitt Trigger	N.A.	2	No
SIO	Input	Y2	3	3.3-V LVTTL	8	2	
SC	Inout	AA1	3	3.3-V LVTTL	8	2	
CS	Output	AB4	3	3.3-V LVTTL	8	2	
Key[0]	Input	H21	6	1.5-V Schmitt Trigger	12	NA	Si
Key[1]	Input	H22	6	1.5-V Schmitt Trigger	12	NA	Si
disp [0]	Output	W16	4	3.3-V LVTTL	8	2	No
disp [1]	Output	AB11	4	3.3-V LVTTL	8	2	No
disp [2]	Output	W15	4	3.3-V LVTTL	8	2	No
disp [3]	Output	AB10	4	3.3-V LVTTL	8	2	No
disp [4]	Output	AA15	4	3.3-V LVTTL	8	2	No
disp [5]	Output	W12	4	3.3-V LVTTL	8	2	No
disp [6]	Output	AA13	4	3.3-V LVTTL	8	2	No
disp [7]	Output	AB12	4	3.3-V LVTTL	8	2	No
mux_disp[0]	Output	Y5	3	3.3-V LVTTL	8	2	No
mux_disp[1]	Output	W6	3	3.3-V LVTTL	8	2	No
mux_disp[2]	Output	W8	3	3.3-V LVTTL	8	2	No
mux_disp[3]	Output	AB8	3	3.3-V LVTTL	8	2	No
mux_disp[4]	Output	R11	3	3.3-V LVTTL	8	2	No
mux_disp[5]	Output	AB6	3	3.3-V LVTTL	8	2	No
mux_disp[6]	Output	AA6	3	3.3-V LVTTL	8	2	No
mux_disp[7]	Output	V10	3	3.3-V LVTTL	8	2	No

5.2 Restricciones de la síntesis

Para el diseño implementado sobre la FPGA **Intel MAX 10 (10M50DAF484C6G)**, se han aplicado las siguientes restricciones de síntesis:

- **Frecuencia de reloj principal:** 50 MHz (definida mediante un *clock constraint* en el archivo `.sdc`).

- **Tiempos de retención y establecimiento:** definidos automáticamente por el *TimeQuest Timing Analyzer* en base a los niveles lógicos de entrada/salida.
- **Asignaciones físicas de pines:** Definidas en el archivo `.qsf` y documentadas en el apartado 5.1.
- **Optimizaciones de síntesis activadas:**
 - Optimización de área.
 - Eliminación de lógica redundante.
 - Registro de señales de salida para mejorar el *timing*.

5.3 Frecuencia máxima de reloj

El diseño cumple sobradamente con los márgenes de temporización establecidos para una frecuencia de 50 MHz, permitiendo incluso operación más rápida si fuera necesario.

6 Prototipado

En este apartado se enumeran las pruebas realizadas con el prototipo y el resultado de cada una de ellas. Todas las pruebas se han realizado en las siguientes condiciones:

- Tarjeta DECA-MAX10 con la tarjeta de expansión conectada.
- Tarjeta alimentada con el alimentador de 5 V DC
- FPGA de la tarjeta configurada con el diseño MEDTH (medt.pof)

Ref.	Descripción	Resultados
PR00	Reset inicial	El circuito la temperatura en grados centígrados y con un periodo de 4 segundos
	Pulsador K0	Pulsamos « rápida » el pulsador K0 para observar que los cambios son coherentes, sin rebotes ni mala multiplexación. A la par medimos el tiempo entre medida y media, y el tiempo de iluminación del display 7 (0 durante 1 segundo cuando mide)
	Pulsador K1	Mismo proceso con el otro pulsador, vemos los cambios en el display 0 según las unidades de medida. Hacemos manualmente el cambio de unidades y comprobamos su valor en la tarjeta (37 C, 310K, 98F)
	Medidas correctas	Dejamos el sistema conectado unos minutos y observamos los cambios de temperatura (la tarjeta se calienta)
	Medidas manuales	Introducimos los valores de -3 grados C y -40 grados C en el registro manualmente para observar su valor en los displays (principalmente la posición del signo negativo).

7 Bibliografía

- [1] . LM71CIMF and SPI bus specification and user manual [online]
<https://www.ti.com/lit/gpn/lm71>
- [2] Tarjeta DECA-MAX10 (página web del fabricante). [online]
<https://www.arrow.com/en/products/deca/arrow-development-tools>
- [3] Tarjeta XDECA. Manual de usuario. [moodle DD2-documentacion técnica]
https://moodle.upm.es/titulaciones/oficiales/pluginfile.php/12137370/mod_resource/content/3/Manual%20XDECA_21.pdf
- [4] DECA Schematic Rev. C. Esquemático de la tarjeta DECA 10M50DAF484C6GES.
[online] [[DECA Schematic Rev.c](#)].
- [5] Intel MAX 10 FPGA Device Datasheet. Datasheet del dispositivo Intel MAX 10
FPGA. [online] [[Intel MAX 10 FPGA Device Datasheet](#)]
- [6] MAX 10 FPGA Device Overview. Descripción general del dispositivo Intel MAX
10 FPGA. [online] [[Intel MAX 10 FPGA Device Overview](#)].
- [7] Intel MAX 10 General Purpose I/O User Guide. Guía del usuario de E/S de
propósito general de Intel MAX 10. [online] [[Intel MAX 10 General Purpose I/O User Guide](#)].